Gramática del Lenguaje LP

Convenciones:
• MAYÚSCULAS: símbolos no terminales.
• ε: cadena vacía.
• id: identificador; num_int: número entero; num_dec: número decimal; texto: constante de texto.
• comp: operador de comparación (> , >= , < , <= , != o ==).

1. Gramática

PROGRAMA	→ LISTA_FUNCIONES PRINCIPAL
LISTA_FUNCIONES	→ DECLARACION_FUNCION LISTA_FUNCIONES | ε
DECLARACION_FUNCION	→ TIPO_RETORNO id (PARAMETROS) BLOQUE
TIPO_RETORNO	→ TIPO | void
TIPO	→ TIPO_BASE DIMENSION
TIPO_BASE	→ char | float | int | boolean
DIMENSION	→ [num_int] DIMENSION | ε
PARAMETROS	→ TIPO id LISTA_PARAMETROS | ε
LISTA_PARAMETROS	→ , TIPO id LISTA_PARAMETROS | ε
PRINCIPAL	→ main ( ) BLOQUE
BLOQUE	→ { LISTA_VARIABLES COMANDOS }
LISTA_VARIABLES	→ TIPO id LISTA_ID ; LISTA_VARIABLES | ε
LISTA_ID	→ , id LISTA_ID | ε
COMANDOS	→ COMANDO COMANDOS | ε
COMANDO	→ LECTURA | ESCRITURA | ASIGNACION | FUNCION | SELECCION | MIENTRAS | RETORNO
LECTURA	→ scanf (TERMINO_LECTURA NUEVO_TERMINO_LECTURA) ;
TERMINO_LECTURA	→ id DIMENSION2
NUEVO_TERMINO_LECTURA	→ , TERMINO_LECTURA NUEVO_TERMINO_LECTURA | ε
DIMENSION2	→ [EXPRESION_ADITIVA] DIMENSION2 | ε
ESCRITURA	→ println (TERMINO_ESCRITURA NUEVO_TERMINO_ESCRITURA) ;
TERMINO_ESCRITURA	→ id DIMENSION2 | CONSTANTE | texto
NUEVO_TERMINO_ESCRITURA	→ , TERMINO_ESCRITURA NUEVO_TERMINO_ESCRITURA | ε
SELECCION	→ if (EXPRESION) BLOQUE SINO
SINO	→ else BLOQUE | ε
MIENTRAS	→ while (EXPRESION) BLOQUE
ASIGNACION	→ id = COMPLEMENTO ;
COMPLEMENTO	→ EXPRESION | FUNCION
FUNCION	→ func id (ARGUMENTOS)
ARGUMENTOS	→ EXPRESION NUEVO_ARGUMENTO | ε
NUEVO_ARGUMENTO	→ , EXPRESION NUEVO_ARGUMENTO | ε
RETORNO	→ return EXPRESION ;
EXPRESION	→ EXPRESION_O
EXPRESION_O	→ EXPRESION_Y EXPRESION_O2
EXPRESION_O2	→ || EXPRESION_Y EXPRESION_O2 | ε
EXPRESION_Y	→ EXPRESION_RELACIONAL EXPRESION_Y2
EXPRESION_Y2	→ && EXPRESION_RELACIONAL EXPRESION_Y2 | ε
EXPRESION_RELACIONAL	→ EXPRESION_ADITIVA EXPRESION_RELACIONAL2
EXPRESION_RELACIONAL2	→ comp EXPRESION_ADITIVA | ε
EXPRESION_ADITIVA	→ EXPRESION_MULTIPLICATIVA EXPRESION_ADITIVA2
EXPRESION_ADITIVA2	→ OPERADOR_ADITIVO EXPRESION_MULTIPLICATIVA EXPRESION_ADITIVA2 | ε

OPERADOR_ADITIVO	→ + | -
EXPRESION_MULTIPLICATIVA	→ FACTOR EXPRESION_MULTIPLICATIVA2
EXPRESION_MULTIPLICATIVA2	→ OPERADOR_MULTIPLICATIVO FACTOR EXPRESION_MULTIPLICATIVA2 | ε
OPERADOR_MULTIPLICATIVO	→ * | / | %
FACTOR	→ SIGNO TERMINO | texto | ! FACTOR | (EXPRESION)
TERMINO	→ id DIMENSION2 | CONSTANTE
SIGNO	→ + | - | ε
CONSTANTE	→ num_int | num_dec

2. Consideraciones para la implementación

- El analizador sintáctico debe recibir los tokens producidos por el analizador léxico; no debe volver a reconocer los caracteres del código fuente.
- Los nombres de los no terminales pueden utilizarse como nombres de funciones del parser en C++.
- Los terminales id, num_int, num_dec y texto deben asociarse con los tipos de token ID, NUM_INT, NUM_DEC y TEXTO del analizador léxico.
- El terminal comp representa los operadores relacionales definidos para LP: >, >=, <, <=, != y ==.
- La producción FUNCION utiliza el terminal func. Este lexema no aparece en la lista de palabras reservadas del PDF léxico anterior; debe incorporarse al lexer.
- La producción RETORNO exige return EXPRESION ;. Por tanto, la gramática no contempla return; sin expresión.
- La producción BLOQUE establece primero LISTA_VARIABLES y después COMANDOS. Por ello, las declaraciones locales deben aparecer antes de los comandos dentro de cada bloque.
- Se conserva la estructura original: LISTA_FUNCIONES aparece antes de PRINCIPAL.

3. Ejemplo de interpretación

Producción: ASIGNACION → id = COMPLEMENTO ;

Interpretación: una asignación está formada por un identificador, el operador de asignación, un complemento y punto y coma.

Ejemplo: edad = 20;

Secuencia conceptual de tokens: ID, =, NUM_INT, ;